

WAPH-Web Application Programming and Hacking

Instructor: Dr. Phu Phung

Student

Name: Aiden Ward

Email: ward3aj@mail.uc.edu

Short-bio: Aiden Ward has a lot of interest in computer science, specifically data science. I also love volleyball and cars.



Figure 1: Aiden's headshot

Repository Information

Repository's URL: <https://github.com/aidenward6/waph-ward3aj/tree/main/waph-ward3aj/labs/hackathon1>

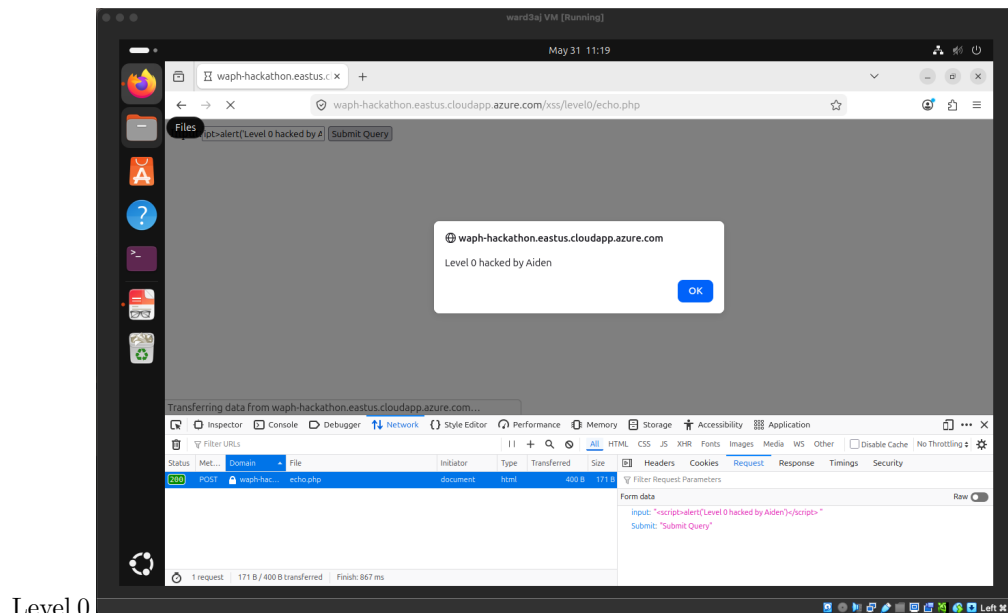
Overview

As this is the first hackathon for this class, I was slightly worried on making sure that I was doing everything correctly. Overall, I think that I satisfied all of the requirements, and I had a great time doing it. This took me the whole day, however I feel like it was very worth it seeing how much I learned in this time. To start off, I was actually hacking a website through multiple different avenues. The cool thing with the website is that there were so many different challenges

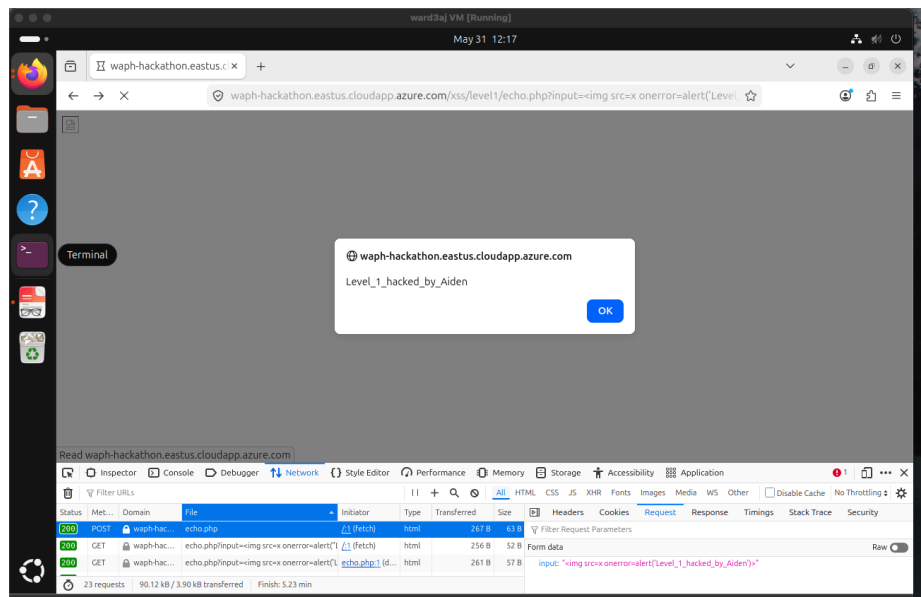
and ways that we had to hack the site. Moving on to task 2, I built on the foundation of Lab 2 to implement robust security measures against Cross-Site Scripting (XSS). I moved from a functional prototype to a hardened one by identifying external input channels and applying strict input validation and output encoding. The primary outcome was learning how to secure asynchronous data requests that made sure that user-supplied data cannot be executed as code in the browser.

Task 1: XSS Attack Simulation

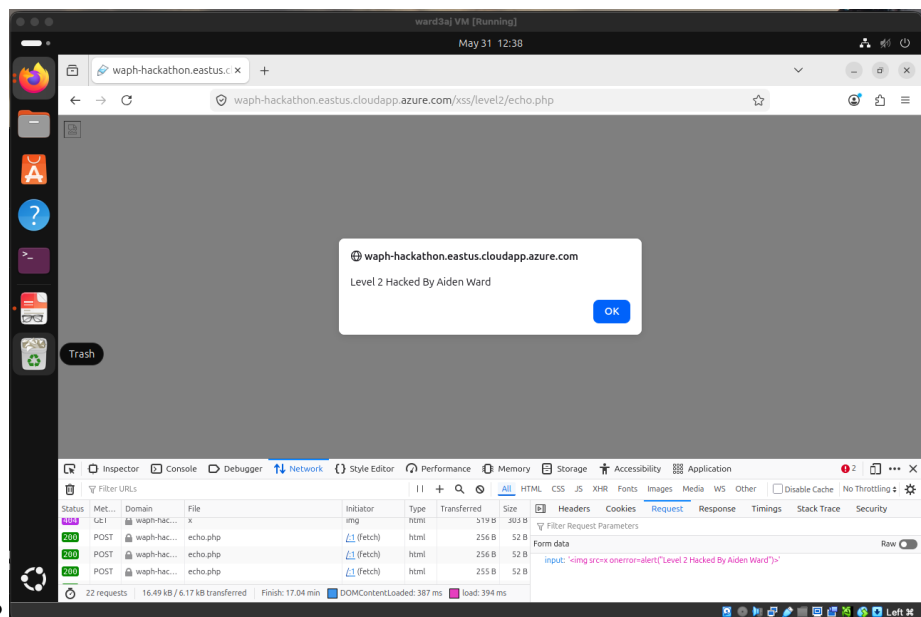
I tested my application's vulnerability by injecting various payloads into the `echo.php` endpoint. Below are the patterns I encountered and the techniques used to bypass or trigger them.



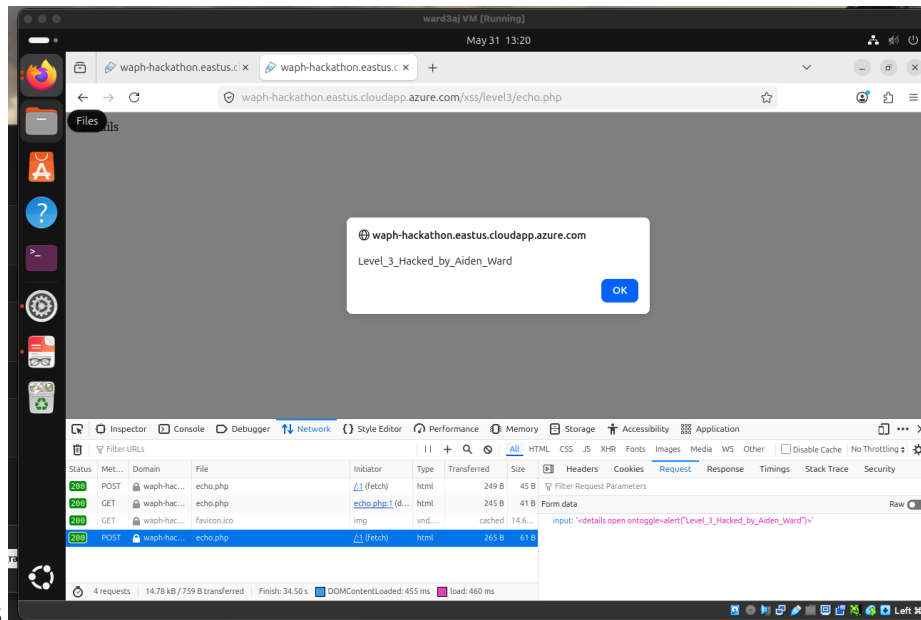
Level 0



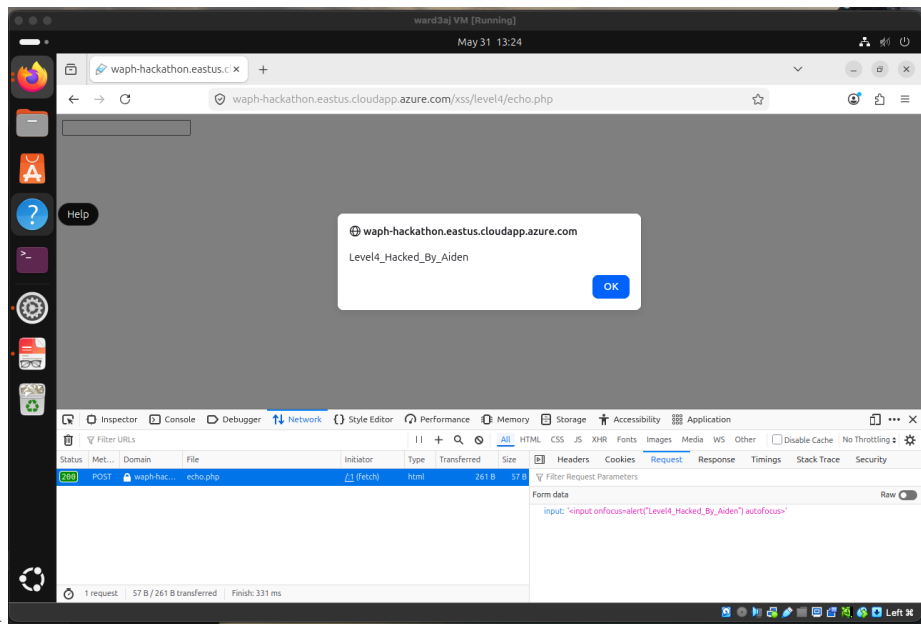
Level 1



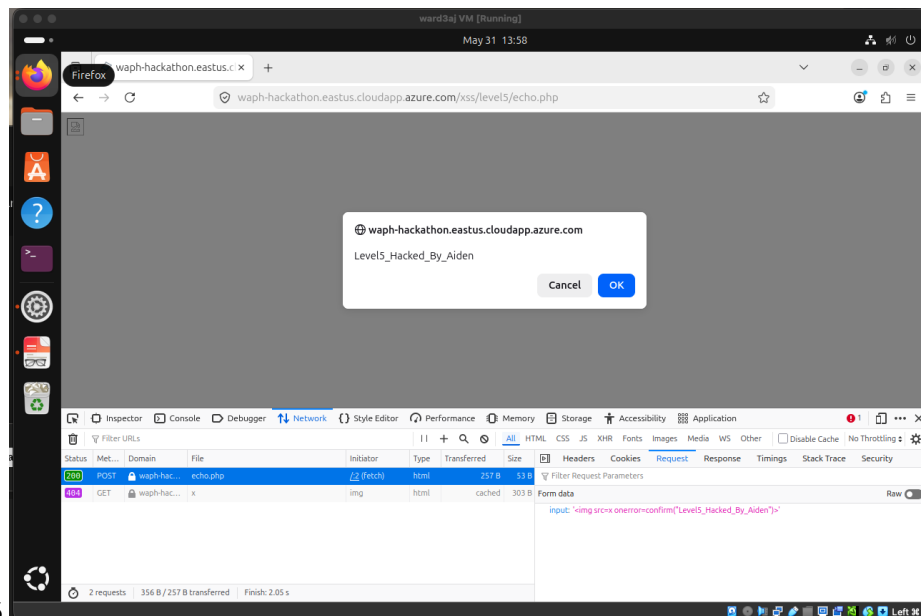
Level 2



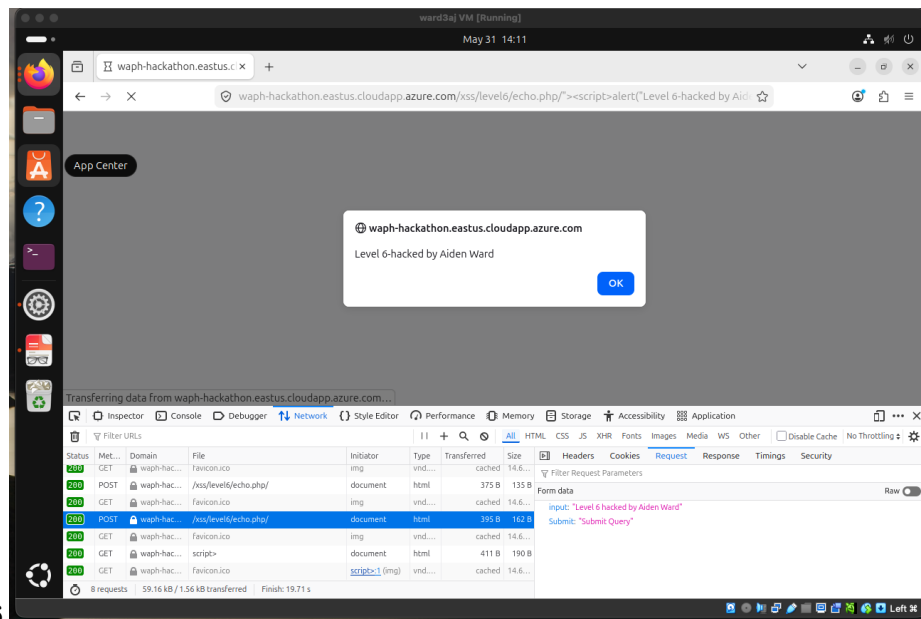
Level 3



Level 4



Level 5



Level 6

Code Guesses

- **Level 2:** The server simply reflected the input.

```
php
$input = $_POST['input'];
echo $input;
```

Level 3: The server stripped specific tags, but I could bypass this with different case sensitivity or alternative tags.

```
$input = $_POST['input'];
$input = str_replace(["<script>", "</script>"], "", $input);
echo $input;
```

Level 4: The server used a keyword blocklist.

```
$input = $_POST['input'];
if (strpos($input, "script") !== false) {
    die("Invalid input");
}
echo $input;
```

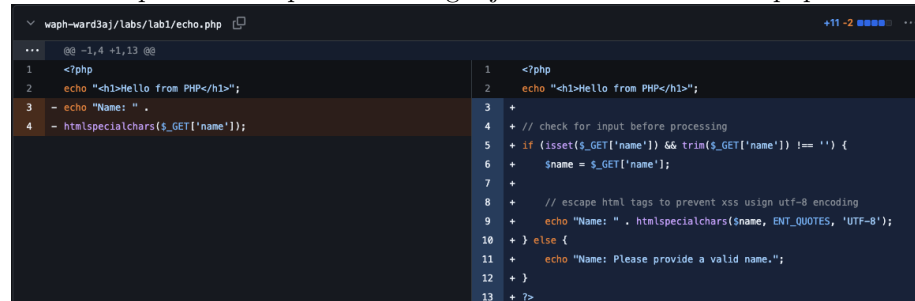
Level 5: Another blocklist implementation.

```
$input = $_POST['input'];
$input = str_replace(["<script>", "</script>", "alert"], "", $input);
echo $input;
```

Level 6: This level used proper encoding, which successfully prevented the XSS attack.

```
$input = $_POST['input'];
$input = htmlspecialchars($input);
echo $input;
```

Task 2: Defenses Implementation? I identified several external input channels, including the AJAX input field and various jQuery-driven inputs. To secure the app, I implemented input validation (to ensure no null or empty strings are processed) and output encoding (to ensure the browser treats input as text rather than HTML). I added htmlspecialchars with ENT_QUOTES and utf8 encoding which acts like a filter so that chars like < and > not able to run which prevnets scripts from being injected Sub-task a: echo.php Revision



```
1 <?php
2 echo "<h1>Hello from PHP</h1>";
3 - echo "Name: " .
4 - htmlspecialchars($_GET['name']);

1 <?php
2 echo "<h1>Hello from PHP</h1>";
3 +
4 + // check for input before processing
5 + if (isset($_GET['name']) && trim($_GET['name']) != '') {
6 +     $name = $_GET['name'];
7 +
8 +     // escape html tags to prevent xss using utf-8 encoding
9 +     echo "Name: " . htmlspecialchars($name, ENT_QUOTES, 'UTF-8');
10 + } else {
11 +     echo "Name: Please provide a valid name.";
12 + }
13 + ?>
```

I updated the server-side code to include input validation and used htmlspecialchars to encode the output. Sub-task b: Front-end Prototype Revision I updated script.js and wash-ward3aj.html to replace insecure .html() methods with .text() and added input validation checks. .text() is very

interesting as it forces the browser to treat all the data thats incomign as literal text, which keeps hackers from running scripts like I did in taks 1.

waph-ward3aj/labs/lab2/script.js	waph-ward3aj/labs/lab2/script.js
@@ -5,11 +5,15 @@ function toggleEmail() {	
5	5
6 function sendAjax() {	6 function sendAjax() {
7 var input = document.getElementById("ajax-input").value;	7 var input = document.getElementById("ajax-input").value;
8 - var xhttp = new XMLHttpRequest();	
9	8
	9 + // validate input is not empty
	10 + if (!input input.trim().length === 0) return;
	11 +
	12 + var xhttp = new XMLHttpRequest();
10 xhttp.onreadystatechange = function() {	13 xhttp.onreadystatechange = function() {
11 if (this.readyState == 4 && this.status == 200) {	14 if (this.readyState == 4 && this.status == 200) {
12 - document.getElementById("ajax-result").innerHTML =	15 + // use textContent to render as plain text, not html
this.responseText;	16 + document.getElementById("ajax-result").textContent =
	this.responseText;
13 }	17 }
14 };	18 };
15	19
waph-ward3aj/labs/lab2/waph-ward3aj.html	
24 <div id="digital-clock">v&v&v	24 <div id="digital-clock">v&v&v
25 <script>	25 <script>
26 function updateClock() {	26 function updateClock() {
27 - document.getElementById("digital-clock").innerHTML = new Date().toLocaleTimeString();	27 + // secure rendering with textContent
28 }	28 + document.getElementById("digital-clock").textContent = new Date().toLocaleTimeString();
29 setInterval(updateClock, 1000);	29 }
30 </script>	30 setInterval(updateClock, 1000);
31	31 </script>
@@ -74,19 +75,26 @@ <body>	
74 <script>	75 <script>
75 function sendAjax() {	76 function sendAjax() {
76 - var input = document.getElementById("ajax-input").value;	77 + var input = document.getElementById("ajax-input").value;
77 - \$.ajax({ url: "/echo.php", type: "GET", data: { data: input }, success: function(res) {	78 + // validate input is not empty
78 - result").html(res); } });	79 + if (!input input.trim().length === 0) return;
	80 + // use .text() for secure rendering
	81 + \$.ajax({ url: "/echo.php", type: "GET", data: { data: input }, success: function(res) {
	82 + result").text(res); } });
79 }	83 }
80	84
81 \$(document).ready(function() {	85 \$(document).ready(function() {
82 - s("#demo-2").click(function() { \$.ajax({ url: "/echo.php", type: "GET", data: { name: s("#query-input-2").val() }, success: function(res) {	86 + s("#demo-2").click(function() {
83 - s("#demo-2").click(function() { \$.ajax({ url: "/echo.php", type: "GET", data: { name: s("#query-input-2").val() }, success: function(res) {	87 + var input = s("#query-input-2").val();
	88 + // validate input is not empty
	89 + if (!input input.trim().length === 0) return;
	90 + // use .text() for secure rendering
	91 + \$.ajax({ url: "/echo.php", type: "GET", data: { name: input }, success: function(res) {
	92 + result-2").text(res); } });
	93 +
	94 + s("#demo-2").click(function() {
	95 + var input = s("#query-input-2").val();
	96 + // validate input is not empty
	97 + if (!input input.trim().length === 0) return;
	98 + // use .text() for secure rendering
	99 + \$.ajax({ url: "/echo.php", type: "GET", data: { name: input }, success: function(res) {
	100 + result-2").text(res); } });
	101 +
100 \$.ajax({ url: "https://x2.jsapi.dev/joke/programming/typing1", success: function(data) {	100 \$.ajax({ url: "https://x2.jsapi.dev/joke/programming/typing1", success: function(data) {
101 - container").text(data.joke); } });	101 - container").text(data.joke); } });
102 }	102 }
103	103
104 function getAge() {	104 function getAge() {
105 - const name = s("#name-input").val();	105 + const name = s("#name-input").val();
106 +	106 + // validate input before fetching
107 +	107 + if (!name name.trim().length === 0) return;
108 fetch("https://api.agify.io/?name="+name) {	108 fetch("https://api.agify.io/?name="+name) {
109 .then(response => response.json())	109 .then(response => response.json())
110 .then(data => { s("#age-result").text("Predicted age for "+data.name);	110 .then(data => { s("#age-result").text("Predicted age for "+data.name);
	111 .then(data => { s("#age-result").text("Predicted age for "+data.name);
	112 }